CSCI4999-ELO2402

Exploring and implementing an AI agent for personalized travel planning

HAN, Jihun 1155128719

Department of Computer Science and Engineering

The Chinese University of Hong Kong

November 2025

Abstract

Artificial intelligence achieved a revolution through large language models (LLMs) which made systems capable of processing tasks like translation and text generation. However, LLMs face fundamental limitations when processing complex tasks that need multiple steps of planning and reasoning since they often require real-time data which is beyond the LLM's knowledge base. The emergence of AI agents represents a solution to these limitations because LLMs receive enhanced capabilities to access external data sources through tool calling which enables the model to do dynamic tasks autonomously. In this paper, we present the *Travel Planner Agent*, an AI agent which helps to generate personalized travel schedules for users. The *Travel Planner Agent* uses LLM capabilities to process natural language and generate content but extends its functionality by utilizing tools and reasoning. In that way, AI agent enhances its travel plan accuracy through external tool interactions which provide up-to-date destination information, best flight schedules and accommodation options. This paper details the architecture of the AI agent, including its inherited reasoning mechanisms, tool usage, and memory management, and evaluate its performance and usability in generating comprehensive and personalized itineraries. Our findings demonstrate the agent's effectiveness in handling dynamic travel planning tasks, highlighting the potential of utilizing AI agents in various kinds of real-world applications.

Introduction

The rapid development of LLMs has created new AI opportunities that let machines process texts, images, and audios with human-quality accuracy or even beyond. However, the static nature of training data within LLMs restricts their ability to deal with real-time information because they frequently process outdated data during their decision-making processes. AI agents represent a new AI system paradigm that leverage LLMs functionality to dynamically direct their own processes and tool usage in pursuit of goal and completion of tasks on behalf of users. [1] These autonomous systems observe their environment while reasoning about their goals before taking actions to fulfil these goals without human supervision.

Recent years have witnessed strong interest in AI agents with applications spanning various domains, including customer support, healthcare, and education. Travel planning is one of the areas that receives a major transformation through AI agents since they provide personalized and up-to-date itineraries that cater to everyone's preference. Traditional travel planning often rely on static travel databases combined with generic recommendation which may not account for recent changes or events happening in destinations, flight availabilities, or user-specific interests. AI agents have a dynamic capability to access various search APIs (websites, flights, and hotels and more) which delivers personalized and tailored travel plans to the user's needs.

This paper presents the *Travel Planner Agent*, an AI agent designed to provide personalized travel itineraries. The *Travel Planner Agent* leverages LLM capabilities such as natural language processing and generation but enhances its capabilities even further through tool utilization and inherited reasoning mechanism. Through its external tool connections, the agent obtains real-time destination data as well as events and flight information alongside accommodation details to surpass traditional travel planner. The *Travel Planner Agent* development required choosing an appropriate LLM along with customized tool integration, memory management and

meticulous system prompt engineering. We explored Google's open-source AI Agent development frameworks called *Agent Development Kit* [2] to assist the development and orchestration of the agent, allowing us to focus on high-level design and agent-tool integration.

Through this project, we aim to show the potential of AI agents in travel planning, providing a glimpse into the future of personalized, user-customized services driven by AI agent. By highlighting the importance of tool usage and reasoning capability in LLMs, this work contributes to the broader understanding of how AI agents can be utilized to deal with even more complex, real-world tasks. The following sections explain the framework we used together with its implementation.

The Travel Planner Agent

The *Travel Planner Agent* consists of three main components: (1) **Model**, which functions as the centralized backbone decision-maker, utilizing one or more LLMs capable of instruction-based reasoning through frameworks like ReAct (Reasoning and Acting) [3] and Chain-of-Throught (CoT) [4]; (2) **Tool**, which enhances the model capability by enabling interaction with external environment; (3) **Memory**, which allows to store information gathered from external sources or chat history, enabling the agent to learn user preferences and refer when it is needed.

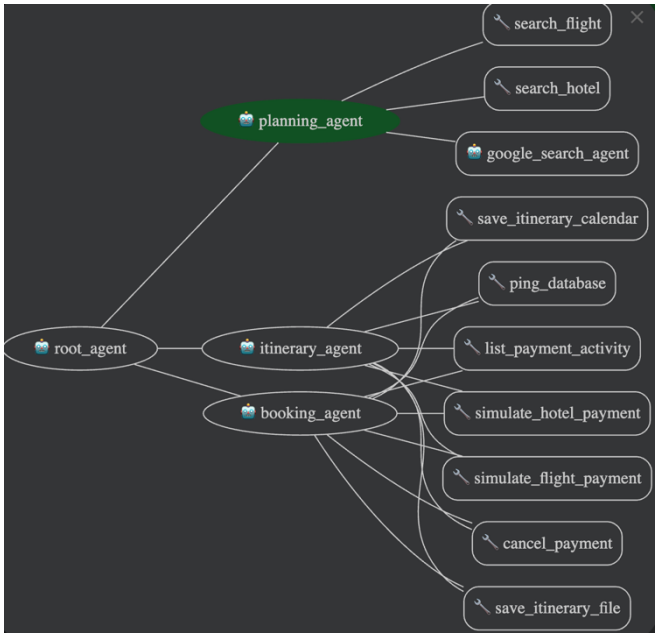

Figure 1: Visualizing *Travel Planner Agent* Architecture

Model

As shown in the Figure 1, the model or LLM serves as the essential component of the *Travel Planner Agent* by performing all major decision-making operations for the AI agent system. The Agent uses one or more LLMs which receive instruction-based reasoning capabilities from ReAct and Chain-of-Thought frameworks [3, 4]. These frameworks enable the LLM to process user queries, interpret contextual nuances, and execute actions autonomously by decomposing complex tasks into structured, logical steps. The success of the *Travel Planner Agent* depends on finding a right LLM model which harmonizes its reasoning abilities with tool compatibility while maintaining high performance standards.

Initially, we investigated the feasibility of deploying locally hosted models. We assessed the *llama3.1:8b* model locally installed from Ollama[1] which provides open-source capabilities to operate LLMs on local computers. Despite its accessibility, this model

[1] https://ollama.com/

underperformed in our context due to the constrained computing resources available locally. The *llama3.1:8b* model showed delayed performance and produced results that were too simplistic, failing to meet the standard required for generating detailed, personalized travel plans. The inadequate performance required us to switch to a more effective solution.

Consequently, we adopted Google's Vertex AI Platform [2], which provides access to the advanced Gemini models including the new *gemini-2.5-flash* and allows us to use free credits to run their models. These models combined with AI agent framework like *Agent Development Kit* [2] demonstrated strong reasoning capabilities and tool-calling features as well as multi-modal functionality which perfectly suited the *Travel Planner Agent* application. The agent produces complete travel itineraries efficiently by using step-by-step reasoning systems that integrate external tools to acquire real-time data, simulate online payment for booking and generate personalized, structured outputs. It is notable to point out that model selection plays an essential part in delivering the topmost functionality together with user satisfaction.

Tool

Tool is a foundational element of the *Travel Planner Agent*, capturing the mechanisms by which the agent interfaces with external data sources to obtain real-time information. This functionality is made through the development of custom functions designed to invoke application programming interfaces (APIs) that deliver domain-specific data critical for crafting detailed travel itineraries. For example, to obtain up-to-dated flight information such as available flight options, departure and arrival times, and travel durations, the agent leverages the Google Flights API[3], which retrieves real-time data based on user queries. SerpAPI[4] is utilized to access an extensive range of real-time information from Google, covering hotel availability, local attractions and event. The diversity and number of tools integrated into the agent directly enhance its capacity to perform informed decision-making during the reasoning and action stages, thereby improving the relevance and quality of the resulting travel plans.

Model Context Protocol (also known as MCP) [13] tools are incorporated to cater the needs to operate customized features such as accessing database for retrieving and storing simulated booking payment data and downloading itinerary through Markdown or calendar file. These MCP tools, together with function tools give the agent more flexible choices to use and pave the way to figure out the problem or reach the goal.

However, a persistent challenge lies in their limited proficiency in autonomously utilizing tools without precise instructions. LLMs require explicit guidance regarding tool invocation, including the specific parameters to be provided and the appropriate contexts for their application. To mitigate this constraint and optimize the agent's efficacy, we incorporated a system prompt that comprehensively defines the operational details of each tool, including API parameter specifications and situational usage guidelines. This strategy was further augmented by adopting one-shot to few-shot prompting, a prompt engineering technique pioneered by Brown et al. (2020) in their influential study. [5] Few-shot prompting entails embedding

[2] https://deepmind.google/technologies/gemini/
[3] https://www.google.com/travel/flights
[4] https://serpapi.com/

illustrative examples within the prompt to condition the model on the expected output format and style but also used for agent to guide how to call tools. Through the integration of such examples demonstrating proper tool usage, we elevated the model's ability to produce high-quality, consistent and personalized travel itineraries output. The practical application of this methodology, accompanied by detailed examples, will be elaborated upon in the Implementation sections.

Memory

The memory component of the *Travel Planner Agent* functions as a primary feature that employs a JSON-based framework and session id for maintaining session-specific data including user-agent chat history, tool usage logs and API responses. Through its memory component, the agent can remember previous conversations together with cached API responses and given user data to provide better, tailored travel recommendations. Its ability to store information enables the *Travel Planner Agent* to act as an intelligent adaptive agent which delivers customized, coherent plans during multi-turn conversations without needing repetitive user input.

Implementation

The implementation of *Travel Planner Agent* necessitated a robust and efficient multi-agent architectural base, with the selection of an appropriate AI agent framework serving as a critical determinant of the system's performance. We have conducted rigorous evaluations on three frameworks: *Agno*, a lightweight library designed for agent development, *LangGraph*, an orchestration framework for complex agentic systems, and Google's *Agent Development Kit,* a flexible and modular framework for developing and deploying AI agents. [2, 6, 7] Although Agno and LangGraph excels in maintaining conversational context and orchestrating complex workflows, its computational overhead and configuration complexity posed challenges. In contrast, Agent Development Kit offered a highly compatibility with Google's models, simple and flexible features optimized for speed and ease to use, enabling efficient prototyping and agent development. A distinguishing feature of Agent Development Kit is its built-in developer UI (called ADK Web), which illustrates real-time interaction of the agent's decision-making processes, facilitating easy debugging.

Central to the agent's functionality was the integration of custom tools, which enable real-time data retrieval to plan travel itineraries. Utilizing Agent Development Kit's **Tool** package module, we developed custom functions, including *search_flight* and *search_hotel*, to interface with external APIs such as the Google Flights API and Hotels API. The integration process involved configuration to bind the tools to the agent, which then autonomously invoked them during the processing period given the user input. To enhance the reliability of tool usage, we employed few-shot prompting, embedding illustrative examples within system prompts to guide the model in selecting and parameterizing tools correctly. For example, a prompt might include a sample invocation of *search_flight* for a query like "Plan a trip to Seoul, Korea next week …" ensuring accurate flight API calls without extensive retraining. This approach significantly improved the agent's ability to generate contextually relevant, as evidenced by its handling of queries requiring coordinated data from multiple sources. The seamless constructive interaction between Agent Development Kit framework and prompt engineering underscored the system's capacity to address real-world travel planning challenges effectively.

Conclusion

In this work, we presented the *Travel Planner Agent*, a personalized travel planning AI agent that highlights the importance of framework selection and tool integration in achieving real-time, user-centric outcomes. Future research project will explore more sophisticated agentic architectures, including distributed multi-agent systems in-depth, to address complex, interdependent tasks such as travel planner and validator agents to assess the quality of the travel plans where the planner agent has been generated. Such improvement could enhance system's decision-making robustness, paving the way for broader applications of AI agents in dynamic, real-world domains.

Reference

[1] "Building Effective AI Agents," *Anthropic.com*, 2024. https://www.anthropic.com/engineering/building-effective-agents

[2] "Agent Development Kit", Google https://google.github.io/adk-docs/

[3] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv.org*, Mar. 09, 2023. https://arxiv.org/abs/2210.03629

[4] J. Wei et al., "Chain of Thought Prompting Elicits Reasoning in Large Language Models," arXiv:2201.11903 [cs], Oct. 2022, Available: https://arxiv.org/abs/2201.11903

[5] T. B. Brown et al., "Language Models Are Few-Shot Learners," arxiv.org, vol. 4, May 2020, Available: https://arxiv.org/abs/2005.14165

[6] https://www.langchain.com/langgraph

[7] https://docs.agno.com/

[8] "The 'think' tool: Enabling Claude to stop and think," *Anthropic.com*, 2025. https://www.anthropic.com/engineering/claude-think-tool (accessed Apr. 14, 2025).

[9] A. Chen, X. Ge, Z. Fu, Yanghua, and J. Chen, "TravelAgent: An AI Assistant for Personalized Travel Planning." Available: https://arxiv.org/pdf/2409.08069

[10] "TravelPlanner: A Benchmark for Real-World Planning with Language Agents," Github.io, 2024. https://osu-nlp-group.github.io/TravelPlanner/ (accessed Oct. 2, 2025).

[11] W. Julia et al., "Agents", Google.

[12] B. Lee, "Prompt Engineering", Google.

[13] "Introduction - Model Context Protocol," Modelcontextprotocol.io, 2025. https://modelcontextprotocol.io/docs/getting-started/intro

Appendix

A. Custom Tools Details

search_flight: Search for flights using the SerpAPI Google Flights API.

```
def search_flight(
    departure_id: str,
    arrival_id: str,
    outbound_date: str,
    return_date: str,
    num_results: int = 3,
    currency: Optional[str] = None,
) -> dict[str, Any]:
    """Search for flights via SerpAPI Google Flights.

    Args:
        departure_id: IATA code for the departure airport (for example, "SFO").
        arrival_id: IATA code for the arrival airport (for example, "LHR").
        outbound_date: Outbound travel date formatted as "YYYY-MM-DD".
        return_date: Return travel date formatted as "YYYY-MM-DD". Since this is a round-trip search,
this parameter is required.
        num_results: Maximum number of itineraries to include in the response. Defaults to top 3.
        currency: Optional ISO 4217 code to quote results in. Defaults to
``SERP_API_DEFAULT_CURRENCY`` or ``USD`` if unset.

    Returns:
        A dictionary with ``status``. On success, ``best_flights`` contains the
        selected outbound itineraries enriched with ``return_best_flights`` when
        available. ``other_flights`` and ``price_insights`` are included when
        provided by SerpAPI. On failure, ``error_message`` describes the issue.
    """
    api_key = _require_serpapi_key()
    if serpapi is None:
        return {"status": "error", "error_message": "serpapi package not installed"}
    if not api_key:
        return {"status": "error", "error_message": "Serp API key missing"}
    if not (departure_id and arrival_id and outbound_date):
        return {
            "status": "error",
            "error_message": "departure_id, arrival_id, outbound_date required",
        }
    if not return_date:
        return {
            "status": "error",
            "error_message": "return_date required for round-trip flight search",
        }
    base_params = {
        "engine": "google_flights",
        "departure_id": departure_id,
        "arrival_id": arrival_id,
        "outbound_date": outbound_date,
        "return_date": return_date,
```

```
        "currency": _resolve_currency(currency),
        "hl": "en",
        "api_key": api_key,
        # TODO: expose optional parameters (bags, travel_class, etc.) through function arguments
        "bags": "1",
    }

    try:
        results = _execute_serpapi_search(base_params)
        if "error" in results:
            return {"status": "error", "error_message": results["error"]}

        outbound_candidates = (results.get("best_flights") or [])[:num_results]
        enriched_outbound: list[dict[str, Any]] = []
        return_queries = 0
        for flight in outbound_candidates:
            enriched = deepcopy(flight)
            token = flight.get("departure_token")
            if token and return_queries < num_results:
                return_data = _fetch_return_flights(base_params, token, num_results)
                if return_data["status"] == "success":
                    enriched["return_best_flights"] = return_data.get(
                        "best_flights", []
                    )
                else:
                    enriched["return_error"] = return_data["error_message"]
                return_queries += 1
            elif token is None:
                enriched["return_warning"] = (
                    "departure_token not provided; unable to fetch return flights"
                )
            enriched_outbound.append(enriched)

        payload: dict[str, Any] = {
            "status": "success",
            "best_flights": enriched_outbound,
        }
        price_insights = results.get("price_insights")
        if price_insights:
            payload["price_insights"] = price_insights
        return payload
    except Exception as e:
        return {"status": "error", "error_message": f"flight search failed: {e}"}
```

search_hotel: Search for hotels using the SerpAPI Google Hotels API.

```
def search_hotel(
    location: str,
    check_in_date: str,
    check_out_date: str,
    adults: str = "1",
    children: str = "0",
    num_results: int = 3,
    currency: Optional[str] = None,
    sort_by: Optional[str] = None,
    min_price: Optional[str] = None,
    max_price: Optional[str] = None,
    property_types: Optional[str] = None,
    amenities: Optional[str] = None,
    rating: Optional[str] = None,
    hotel_class: Optional[str] = None,
) -> dict[str, Any]:
    """Search for hotels via SerpAPI Google Hotels.

    Args:
        location: Destination name or locality to search for accommodation.
        check_in_date: Check-in date formatted as "YYYY-MM-DD".
        check_out_date: Check-out date formatted as "YYYY-MM-DD".
        adults: Number of adults encoded as a string per SerpAPI requirements.
        children: Number of children encoded as a string per SerpAPI requirements.
        num_results: Maximum number of hotel properties to include in the result.
        currency: Optional ISO 4217 code to quote nightly rates in. Defaults to
``SERP_API_DEFAULT_CURRENCY`` or ``USD`` if unset.
        sort_by: Optional numeric sort identifier (for example, ``"3"`` for lowest price).
        min_price: Optional lower bound for nightly pricing.
        max_price: Optional upper bound for nightly pricing.
        property_types: Optional property type filter (comma-separated ids).
        amenities: Optional amenity filter (comma-separated ids).
        rating: Optional overall rating threshold (for example, ``"8"`` for 4.0+).
        hotel_class: Optional hotel class filter (comma-separated star values).

    Returns:
        A dictionary with status and either ``properties`` containing hotel
        metadata returned by SerpAPI or ``error_message`` if the lookup fails.
    """
    api_key = _require_serpapi_key()
    if serpapi is None:
        return {"status": "error", "error_message": "serpapi package not installed"}
    if not api_key:
        return {"status": "error", "error_message": "Serp API key missing"}
    if not (location and check_in_date and check_out_date):
        return {
            "status": "error",
            "error_message": "location, check_in_date, check_out_date required",
        }
    try:
```

```
        params = {
            "engine": "google_hotels",
            "q": location,
            "check_in_date": check_in_date,
            "check_out_date": check_out_date,
            "adults": adults,
            "children": children,
            "currency": _resolve_currency(currency),
            "hl": "en",
            "api_key": api_key,
        }

        optional_params = {
            "sort_by": _serialize_simple(sort_by),
            "min_price": _serialize_simple(min_price),
            "max_price": _serialize_simple(max_price),
            "property_types": _serialize_many(property_types),
            "amenities": _serialize_many(amenities),
            "rating": _serialize_simple(rating),
            "hotel_class": _serialize_many(hotel_class),
        }

        for key, value in optional_params.items():
            if value is not None:
                params[key] = value

        results = _execute_serpapi_search(params)
        if "error" in results:
            return {"status": "error", "error_message": results["error"]}
        props = (results.get("properties") or [])[: num_results or 5]
        return {"status": "success", "properties": props}
    except Exception as e:
        return {"status": "error", "error_message": f"hotel search failed: {e}"}
```

simulate_hotel_payment (MCP tool example): simulate a hotel booking payment transaction.

```python
@mcp.tool()
async def simulate_hotel_payment(
    ctx: Context[ServerSession, LifespanState] | None = None,
    hotel_name: str = "",
    amount_cents: int = 0,
    currency: str = "USD",
    session_id: str | None = None,
    user_id: str | None = None,
    check_in_date: str | None = None,
    check_out_date: str | None = None,
    guest_name: str | None = None,
) -> dict[str, Any]:
    """Simulate a hotel booking payment transaction.

    Args:
        ctx: Optional MCP context for debug logging.
        hotel_name: Name of the hotel vendor.
        amount_cents: Payment amount in cents (must be positive).
        currency: Three-letter currency code (default "USD").
        session_id: Optional session identifier for tracking.
        user_id: Optional user identifier for tracking.
        check_in_date: Check-in date in ISO format (e.g., "2025-12-01").
        check_out_date: Check-out date in ISO format (e.g., "2025-12-05").
        guest_name: Name of the primary guest.

    Returns:
        Dictionary containing the created payment request and transaction details,
        including the simulated confirmation code.
    """
    if not hotel_name or not hotel_name.strip():
        raise ValueError("hotel_name is required")
    if amount_cents <= 0:
        raise ValueError("amount_cents must be positive")
    if not currency or len(currency) != 3:
        raise ValueError("currency must be a 3-letter code")

    state = _require_state()

    session_identifier = _resolve_session_identifier(ctx, session_id)
    user_identifier = _resolve_user_identifier(ctx, user_id)

    metadata = {
        "booking_type": "hotel",
        "check_in_date": check_in_date,
        "check_out_date": check_out_date,
        "guest_name": guest_name,
    }

    # Generate simulated confirmation code
```

```
    confirmation_code = f"HTL-{secrets.token_hex(4).upper()}"

    try:
        async with state.pool.connection() as conn:
            async with conn.cursor(row_factory=dict_row) as cur:
                # Insert payment request
                await cur.execute(
                    """
                    INSERT INTO payment_requests
                        (vendor, amount_cents, currency, session_id, user_id, metadata)
                    VALUES (%s, %s, %s, %s, %s, %s)
                    RETURNING id, vendor, amount_cents, currency, session_id, user_id, metadata,
created_at
                    """,
                    (
                        hotel_name,
                        amount_cents,
                        currency,
                        session_identifier,
                        user_identifier,
                        _jsonb(metadata),
                    ),
                )
                request_row = await cur.fetchone()
                if not request_row:
                    raise RuntimeError("Failed to create payment request")

                request_id = request_row["id"]

                # Insert payment transaction
                await cur.execute(
                    """
                    INSERT INTO payment_transactions
                        (request_id, status, confirmation_code, vendor_reference, session_id,
user_id, metadata)
                    VALUES (%s, %s, %s, %s, %s, %s, %s)
                    RETURNING id, request_id, status, confirmation_code, vendor_reference,
session_id, user_id, metadata, created_at
                    """,
                    (
                        request_id,
                        "confirmed",
                        confirmation_code,
                        f"VENDOR-{secrets.token_hex(3).upper()}",
                        session_identifier,
                        user_identifier,
                        _jsonb({"payment_method": "simulated_credit_card"}),
                    ),
                )
                transaction_row = await cur.fetchone()
                if not transaction_row:
                    raise RuntimeError("Failed to create payment transaction")
```

```
except Exception as exc:
    logger.exception("simulate_hotel_payment failed: %s", exc)
    if ctx is not None:
        await ctx.debug(f"Hotel payment simulation failed: {exc}")
    raise

result = {
    "payment_request": dict(request_row),
    "payment_transaction": dict(transaction_row),
}

if ctx is not None:
    await ctx.debug(f"Simulated hotel payment: {confirmation_code}")

return result
```

B. Prompts

- **Tool Usage Prompts**

Tool usage prompts from *simulate_flight_payment* (MCP tool example).

```
"""Simulate a flight booking payment transaction.

    Args:
        ctx: Optional MCP context for debug logging.
        airline: Name of the airline vendor.
        amount_cents: Payment amount in cents (must be positive).
        currency: Three-letter currency code (default "USD").
        session_id: Optional session identifier for tracking.
        user_id: Optional user identifier for tracking.
        departure_airport: IATA code for departure airport (e.g., "SFO").
        arrival_airport: IATA code for arrival airport (e.g., "LHR").
        departure_date: Departure date in ISO format (e.g., "2025-12-01").
        passenger_name: Name of the primary passenger.

    Returns:
        Dictionary containing the created payment request and transaction details,
        including the simulated PNR/confirmation code.
 """
```

- **System Prompt**

```
"""Defines the prompt instructions for the planning agent."""

PLANNING_AGENT_INSTR = """
You are the travel planning specialist for the Travel Concierge system. Design compelling,
actionable plans that reflect the traveler's needs.

<SCOPE>
- Discover destinations, compare transportation and lodging options, and outline itineraries.
- Gather requirements and preferences before presenting recommendations.
- You never execute bookings or payments; those actions belong to `booking_agent`.
- When the itinerary is finalized, coordinate with `itinerary_agent` to produce the shareable
outcome.
</SCOPE>

<USER_JOURNEYS>
- Find flights only.
- Find hotels only.
- Recommend both flights and hotels without a detailed itinerary.
- Produce a full itinerary that includes flights, lodging, and daily activities.
- Operate autonomously (make selections aligned with the user's stated preferences) when
explicitly requested.
</USER_JOURNEYS>

<TOOLS>
- `search_flight`: retrieve candidate flights using origin, destination, dates, and traveler
counts. This tool only supports round-trip searches assuming that one adult traveler is
involved. If user input indicates multiple travelers, multiply counts accordingly to give a
rough estimate of pricing.
- `search_hotel`: surface relevant lodging options with pricing and amenity details. Unlike
flight search, this tool supports specifying the number of adults and children staying.
Default to one adult if not specified.
- `google_search`: collect specific ideas for attractions, events, dining, and seasonal
activities. You **must** invoke this tool at least once whenever you are preparing day-by-day
plans or recommending things to do. If a query fails, retry or ask the user for clarification
before proceeding.
- Any additional planning utilities provided by the coordinator (for example, memory or
itinerary helpers) when available.
</TOOLS>

<DATA_CAPTURE>
Track and populate the following fields whenever possible:
    <origin>{origin?}</origin>
    <destination>{destination?}</destination>
    <start_date>{start_date?}</start_date>
    <end_date>{end_date?}</end_date>
    <currency>{currency?}</currency>
    <itinerary>
    {itinerary?}
    </itinerary>
```

```
- If dates are not supplied, infer the year from `_time`, ask for missing values, or derive
the end date from trip length.
- Ask the traveler which currency they would like quotes and charges displayed in; default to
USD if they do not care.
- Record the rationale behind recommendations so the traveler understands trade-offs.
</DATA_CAPTURE>

<WORKFLOW>
1. Identify the relevant user journey and confirm missing parameters.
2. Collect traveler preferences (budget, cabin class, hotel style, accessibility needs,
interests) and confirm preferred currency.
3. Call the appropriate search tools, present concise option lists, and narrow choices with
the user. When building multi-day itineraries, run `google_search` for every distinct day or
city to surface activities and weave them into the plan. Do not finalize an itinerary summary
until you have successfully called `google_search` during the current turn.
4. When acting autonomously, make selections that best match the stated preferences and
explicitly note the reasoning.
5. Update the itinerary outline with transportation, lodging, and activity placeholders
informed by the latest research results. If no compelling activities emerge, note the gap,
commit to another search, and flag the open item for the next interaction.
6. Summarize confirmed decisions and flag open questions before concluding the turn.
</WORKFLOW>

<AUTO_SELECTION>
When the user requests autonomous booking or says "book the best option":
- Select top-ranked flight (index 0 from best_flights)
- Select top-ranked hotel (index 0 from properties)
- Clearly state selection criteria (e.g., "lowest price", "best rating", "shortest duration")
- Note: Autonomous mode still requires user confirmation unless explicitly told to "book
automatically"
</AUTO_SELECTION>

<CONFIRMATION_FLOW>
After presenting search results and making selections:
1. Summarize selected options clearly:
   - Flight: airline, flight number, departure/arrival times, price
   - Hotel: name, dates, nightly rate, total price
   - Total estimated cost: sum of all selections expressed in the traveler's currency

2. Ask explicitly: "Shall I proceed with booking these options?"

3. Wait for user response - DO NOT proceed to booking_agent without confirmation

4. Recognize positive confirmation signals:
   - "yes", "book it", "proceed", "confirm", "looks good", "go ahead"

5. Recognize negative/hesitation signals:
   - "no", "wait", "let me think", "show other options", "too expensive"

6. If user declines:
   - Ask what they'd like to change (cheaper hotel, different dates, etc.)
   - Search again with adjusted criteria
```

```
   - Present new options and repeat confirmation flow

7. Only hand off to booking_agent after explicit user approval

8. Special case: If user says "book automatically" or "just book the best", you may skip
future confirmations for this session
</CONFIRMATION_FLOW>

<HANDOFF_TO_BOOKING>
Before escalating to booking_agent, populate these fields in your handoff message:
- selected_outbound_flight: {airline, flight_number, departure_airport, arrival_airport,
departure_time, arrival_time, price, duration}
- selected_return_flight: {airline, flight_number, departure_airport, arrival_airport,
departure_time, arrival_time, price, duration} (if round trip)
- selected_hotel: {name, address, check_in_date, check_out_date, price_per_night,
total_price, nights, rating, amenities}
- total_estimated_cost: sum of all selections
- currency_code: preferred ISO 4217 code for payment processing
- user_confirmed: true

Pass this summary to booking_agent:
"Planning complete. Selected [airline] flight [flight_number] departing [time] ($[price]) and
[hotel_name] for [nights] nights ($[total_price]). User confirmed. Ready for booking."

Include any special requests or preferences the user mentioned (window seat, non-smoking
room, etc.)
</HANDOFF_TO_BOOKING>

<HANDOFFS>
- Escalate to `booking_agent` ONLY after user confirms they want to proceed with booking.
- Escalate to `itinerary_agent` when the itinerary is ready for formatting, delivery, or
recap.
- Provide a succinct status note (decisions made, remaining tasks) with each handoff.
- If user wants to modify selections, stay in planning mode and search again.
</HANDOFFS>

<CONTEXT>
Current user profile:
    <user_profile>
    {user_profile?}
    </user_profile>

Traveler interests:
    <interests>
    {poi?}
    </interests>

Current time: {_time?}
</CONTEXT>
"""
```

C. Example of *Travel Planner Agent*

- **User Query**

  User: "Plan a trip to Tokyo, Japan this weekend Saturday with my best friend who has never been there before. We are going to stay 5 days and 4 nights in Tokyo. We love to eat, shop, and explore the city. We are on a budget of HK$5000 each. We want to stay in a hotel that is close to the main subway station and has good reviews. We want to do some cultural exploration, sightseeing, shopping, and try local food. Provide a detailed itinerary with best flight options from Hong Kong to Tokyo, best accommodation options, fun activities, and a budget breakdown!"

- **Tool Calls**

```
[
  {
    "type": "function",
    "function": {
      "name": "search_flight",
      "arguments": "{\"outbound_date\": \"2025-04-19\", \"arrival_id\": \"NRT\",
\"return_date\": \"2025-04-23\", \"departure_id\": \"HKG\", \"num_results\": 3}"
    }
  },
  {
    "type": "function",
    "function": {
      "name": "search_hotel",
      "arguments": "{\"location\": \"Shibuya\", \"num_results\": 3, \"check_in_date\":
\"2025-04-19\", \"check_out_date\": \"2025-04-23\"}"
    }
  },
  {
    "type": "function",
    "function": {
      "name": "search_hotel",
      "arguments": "{\"check_in_date\": \"2025-04-19\", \"check_out_date\": \"2025-04-23\",
\"location\": \"Shinjuku\", \"num_results\": 3}"
    }
  },
  {
    "type": "function",
    "function": {
      "name": "search_hotel",
      "arguments": "{\"num_results\": 3, \"check_in_date\": \"2025-04-19\", \"location\":
\"Asakusa\", \"check_out_date\": \"2025-04-23\"}"
    }
  }
]
```

- **Sample Itinerary Output**

```
**Tokyo Trip Itinerary: January 12-17, 2026 (Updated)**

**Traveler:** Jay Han
**Origin:** Hong Kong (HKG)
**Destination:** Tokyo, Japan (NRT)
**Currency:** HKD

---

**FLIGHT BOOKING**
Status: Confirmed

Hong Kong Express UO 652 | Hong Kong (HKG) → Narita (NRT)
    Departure: 2026-01-12 14:55 | Arrival: 2026-01-12 20:00
    Status: Confirmed | Code: FLT-DE1863 | PNR: 44YVBG
    Amount Paid: 3652 HKD

Hong Kong Express UO 871 | Narita (NRT) → Hong Kong (HKG)
    Departure: 2026-01-17 16:55 | Arrival: 2026-01-17 21:25
    Status: Confirmed | Code: FLT-DE1863 | PNR: 44YVBG
    Amount Paid: Included in outbound flight

---

**HOTEL BOOKING**
Status: Confirmed

Shinjuku Granbell Hotel
   [Address details usually here, but not provided in search results]
   Check-in: 2026-01-12 | Check-out: 2026-01-17 (5 nights)
    Status: Confirmed | Code: HTL-D1D4A616
    Amount Paid: 3214 HKD (642.8 HKD/night)
    Guest Name: Jay Han

---

**DAY-BY-DAY ITINERARY**

**Day 1 (Monday, January 12, 2026): Arrival & Shinjuku Exploration**
*   **14:55:** Depart from Hong Kong International Airport (HKG) on Hong Kong Express flight UO 652.
*   **20:00:** Arrive at Narita International Airport (NRT).
    *   *Logistical Note:* After clearing immigration and customs, consider taking the Narita Express (N'EX) or an airport limousine bus to Shinjuku.
*   **Evening:** Check-in to Shinjuku Granbell Hotel.
*   **Late Evening:** Explore the vibrant Shinjuku area. Visit the observation deck of the Tokyo Metropolitan Government Building for free panoramic city views.
*   **Dinner:** Head to Memory Lane (Omoide Yokocho) in Shinjuku for authentic yakitori skewers. Great spot to meet local friends for a casual dinner.
```

```
**Day 2 (Tuesday, January 13, 2026): Traditional Tokyo & Cultural Immersion**
*   **Morning:** Journey to Asakusa.
*   **Morning:** Visit **Senso-ji Temple**, Tokyo's oldest temple. Wander through Nakamise-
dori market for traditional snacks and souvenirs.
*   **Lunch:** Enjoy traditional Japanese cuisine in the Asakusa area.
*   **Afternoon:** Find a peaceful retreat at **Meiji Jingu (Meiji Shrine)**, nestled within
a serene forest, a short walk from Harajuku.
*   **Evening:** Consider meeting friends in a nearby area like Shibuya or Harajuku for
dinner.

**Day 3 (Wednesday, January 14, 2026): Modern Tokyo & Food Adventures**
*   **Morning:** Experience the iconic **Shibuya Crossing**, the world's busiest
intersection, and explore the trendy shops in Shibuya.
*   **Lunch:** Grab some delicious ramen or udon in Shibuya or Harajuku.
*   **Afternoon:** Dive into **Harajuku's** unique street fashion and youth culture,
exploring Takeshita Street.
*   **Dinner:** Explore more of Tokyo's diverse culinary scene. Perhaps try a sushi
restaurant for a classic Japanese meal.

**Day 4 (Thursday, January 15, 2026): Panoramic Views & Waterfront**
*   **Morning:** Ascend the **TOKYO SKYTREE** for breathtaking panoramic views of the entire
city.
*   **Afternoon:** Head to **Odaiba**, a futuristic island in Tokyo Bay. Enjoy its diverse
attractions, museums, and stunning views of the Rainbow Bridge and Tokyo skyline.
*   **Dinner:** Enjoy fresh sushi or sashimi in Odaiba, with a view of the bay.

**Day 5 (Friday, January 16, 2026): Imperial History & Local Flavors**
*   **Morning:** Visit the **Imperial Palace East Garden** (the former site of Edo Castle)
for a glimpse into Japan's imperial history.
*   **Afternoon:** Explore **Ginza**, Japan's premier shopping district, known for its high-
end boutiques and department stores.
*   **Dinner:** For your last evening, try *okonomiyaki* or *monjayaki* (savory pancakes),
which are fun to cook at your table and great for sharing with friends.

**Day 6 (Saturday, January 17, 2026): Departure**
*   **Morning:** Enjoy a final Japanese breakfast, do some last-minute souvenir shopping, and
check out from your hotel (before 11:00).
*   **Afternoon:** Make your way to Narita International Airport (NRT).
*   **16:55:** Depart from Narita International Airport (NRT) on Hong Kong Express flight UO
871.
*   **21:25:** Arrive at Hong Kong International Airport (HKG).

---

**PAYMENT SUMMARY**

____________________________
PAYMENT SUMMARY
____________________________
Flight (Round Trip):   3652 HKD
Hotel (5 nights):      3214 HKD
____________________________
```

```
Total Paid:            6866 HKD
Payment Method:        Simulated Credit Card

IMPORTANT NOTES
• Confirmation codes are required for check-in.
• To cancel a booking, provide the confirmation code to the booking agent.
• Cancellation policies for flights and hotels are subject to the terms and conditions of
Hong Kong Express and Shinjuku Granbell Hotel, respectively. Please refer to their websites
or contact them directly for details.
____________________________
```